# Firefighting Drone Swarm Report

Group A4 -7

Aryan Kumar Singh (101305235)

Abdullah Khan (101299776)

Kevin Abeykoon (101301971)

SYSC3303A

April 08, 2026

# Table Of Contents

**Breakdown of Responsibilities for each Iteration** ..... **3**

1. Aryan Kumar Singh (101299776) ..... 3

2. Kevin Abeykoon (101301971) ..... 4

3. Abdullah Khan (101305235) ..... 5

**Diagrams:** ..... **7**

UML Class Diagrams: ..... 7

State Diagrams: ..... 8

Drone Subsystem: ..... 8

Scheduler: ..... 9

Sequence Diagrams: ..... 10

Entire Program ..... 10

Fire Subsystem: ..... 11

Drone Subsystem ..... 12

**Detailed Setup Instructions** ..... **13**

**Results From Measurements** ..... **14**

**Reflection** ..... **15**

# Breakdown of Responsibilities for each Iteration

## 1. Aryan Kumar Singh (101299776)

**Iteration 1**

- Worked on the implementation of all 9 classes including the GUI
- Worked on the Javadoc documentation

**Iteration 2**

- Improved GUI for clearly tracking of the fire and drone states with colour, animation, and a table.
- Integrated a state machine design for scheduler and drones (i.e. classic switch case's implementation)
- Worked on some Javadoc documentation

**Iteration 3**

- Helped implement UDP communication between all three subsystems.
- UML Class Diagram

**Iteration 4**

- Visualized hard and soft fault states and events on the GUI
- Seperated drone system into a subsystem that receives UDP messages and relays its contents and individual drone machines that are receive orders from the subsystem and not the Scheduler directly.
- UML Class Diagram

**Iteration 5**

- Added variable zone sizing but enabling user to input a file specifying rectangular zones of any number if they fulfilled the zone requirements.
- Did tests on the dynamic zones and drone agent capacity

## 2. Kevin Abeykoon (101301971)

**Iteration 1**

- Worked on the implementation of Drone State, DroneSubsystem, FireEvent, and Scheduler
- Worked on Javadoc documentation for all classes and README

**Iteration 2**

- Wrote the Scheduler tests
- Created the Sequence diagram depicting how the GUI retrieves data and redisplays, aswell as the general flow of the simulation
- Since I did most of the Scheduler work for this iteration during the last iteration, there was not much more to do
- In the last iteration, I created a multiple drone scheduling queue with "logic" as per the project specifications
- Worked on some Javadoc documentation

**Iteration 3**

- Made some Scheduler changes
- Wrote the UDP testing

**Iteration 4**

- Implemented the fault handling system in DroneSubsystem, FireIncidentSubsystem, and Scheduler
- I majorily did this by adding to the state based system: adding states for hard fault, entering a soft fault, and recovering from a soft fault

**Iteration 5**

- Implemented the battery recharge mechanic
- Implemented the battery recharge mechanic's representation in the GUI
- Implemented the battery recharge tests

## 3. Abdullah Khan (101305235)

**Iteration 1**

- Worked on the implementation of DroneSubsystem
- Worked on UML Sequence Diagrams

**Iteration 2**

- Worked on DroneSubsystem Tests (for both iterations)
- Made State Machine Diagram for DroneSubSystem

**Iteration 3**

- Reworked DroneSubsystem and Scheduler
- Updated DroneSubsystem state machine diagram

**Iteration 4**

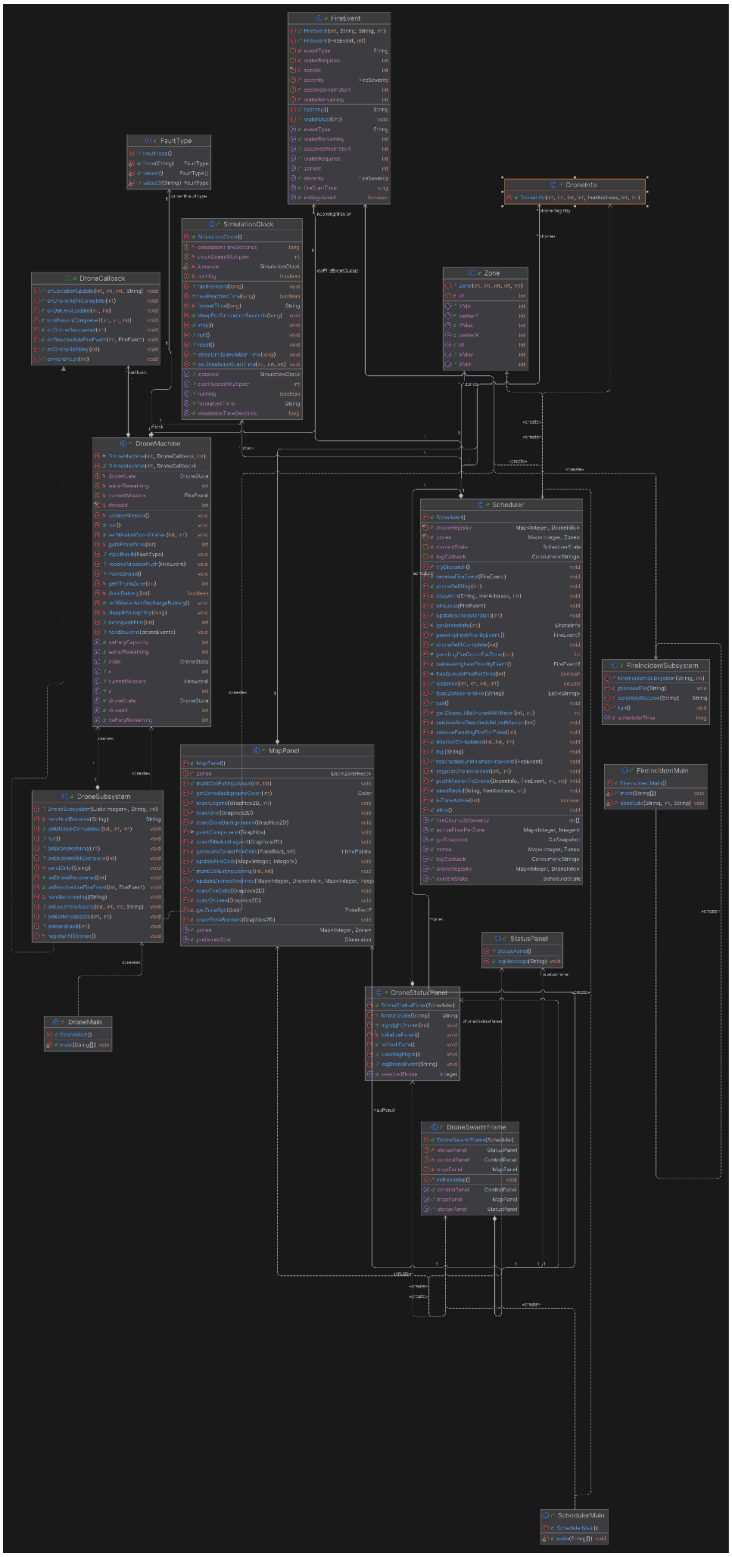
- Worked on Unit Tests

**Iteration 5**

- Worked on EventLogger and Metrics Analysis

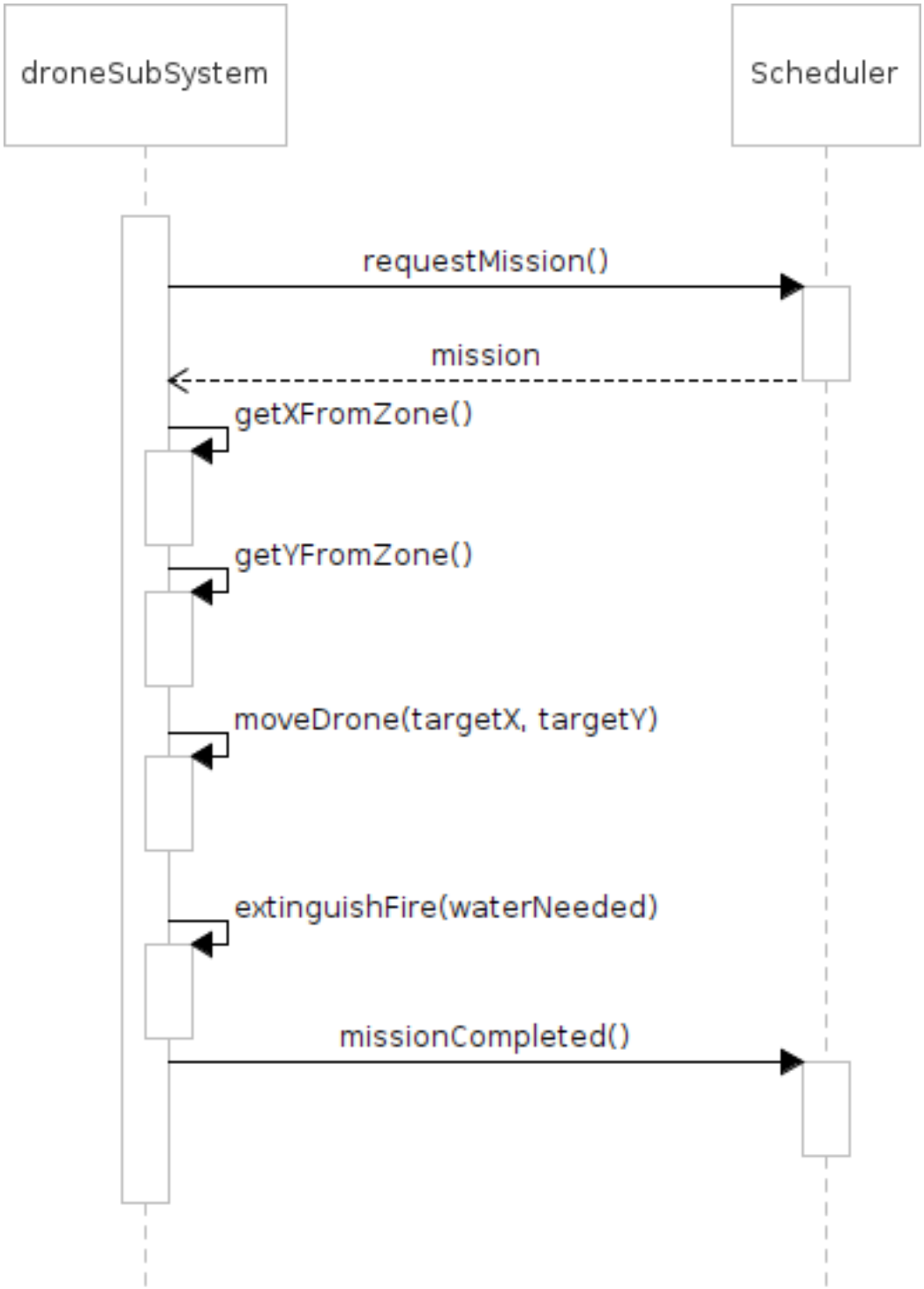
# Diagrams:

## UML Class Diagrams:

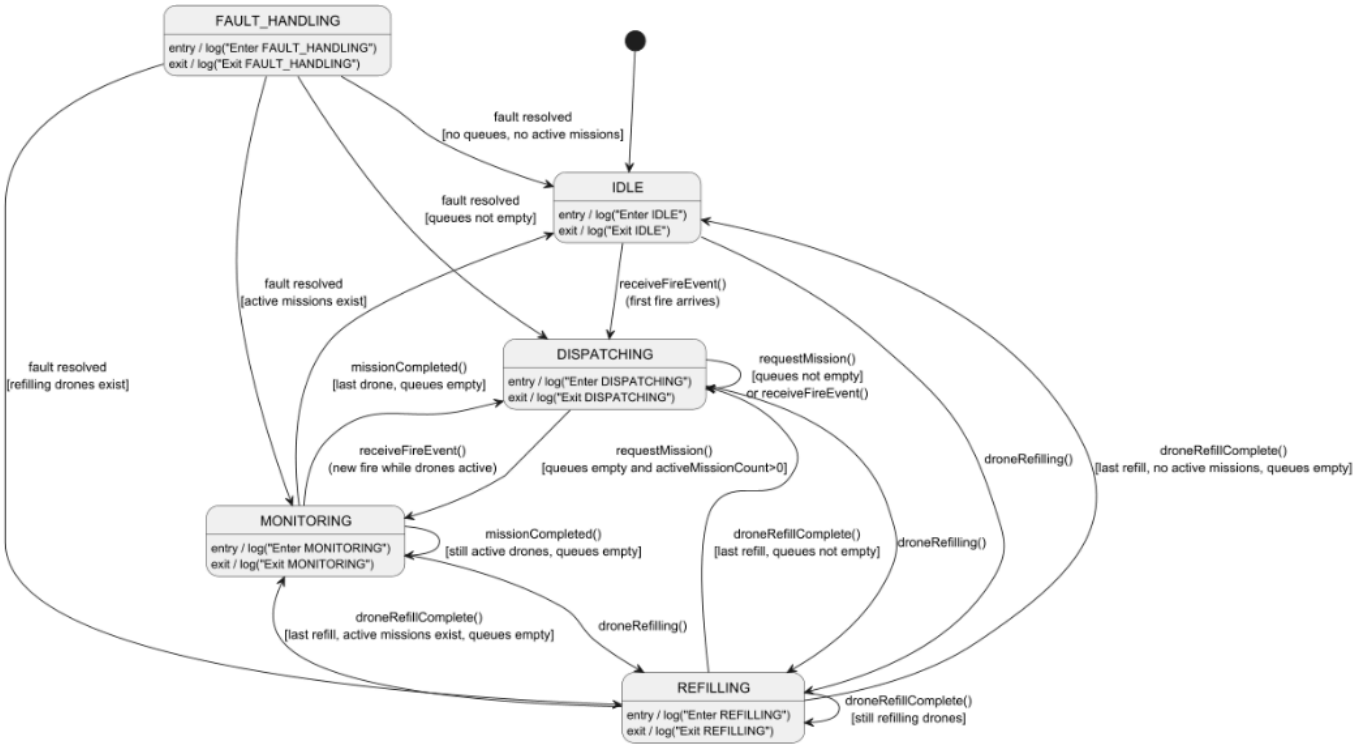


# State Diagrams:

Drone Subsystem:

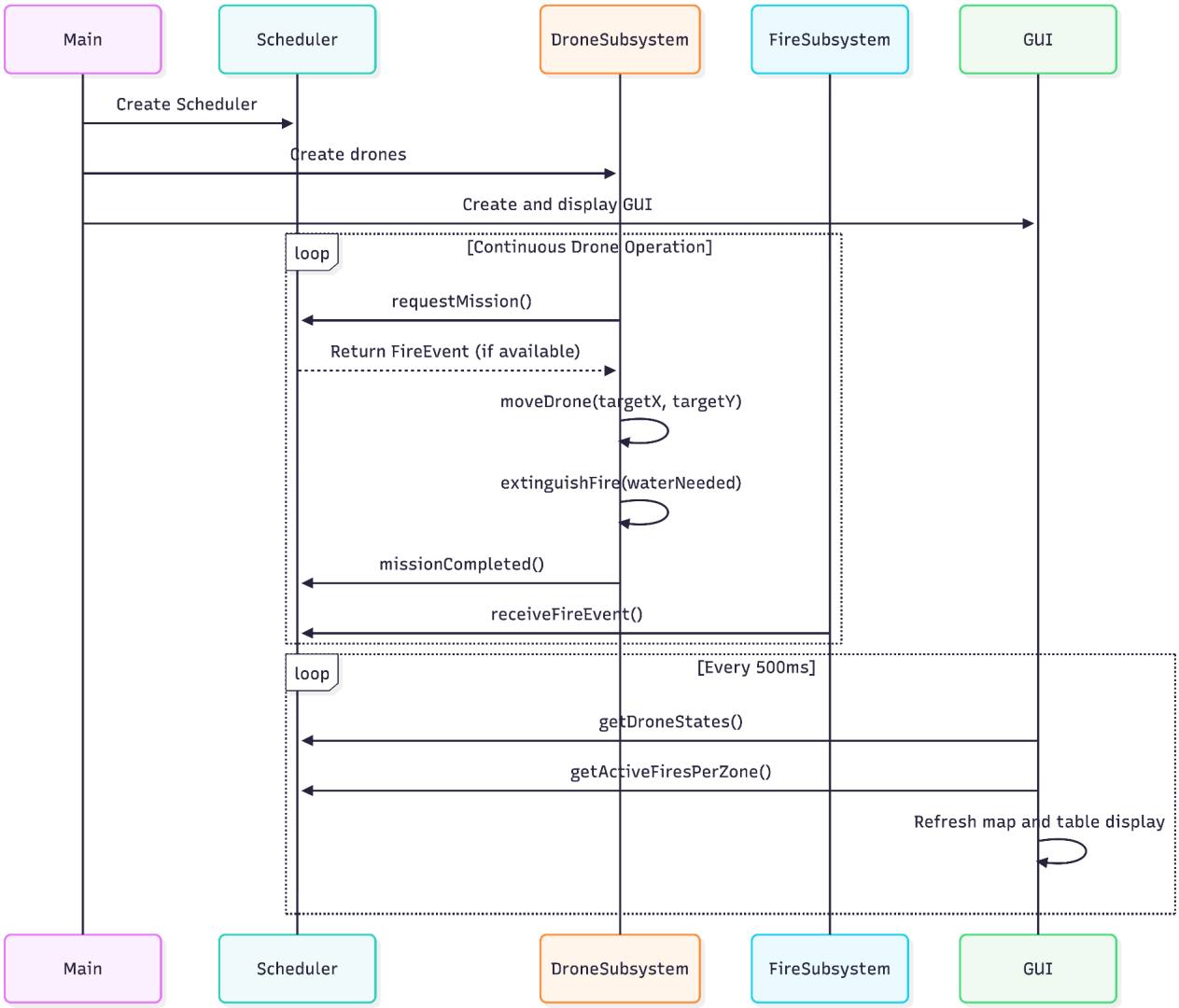


Scheduler:

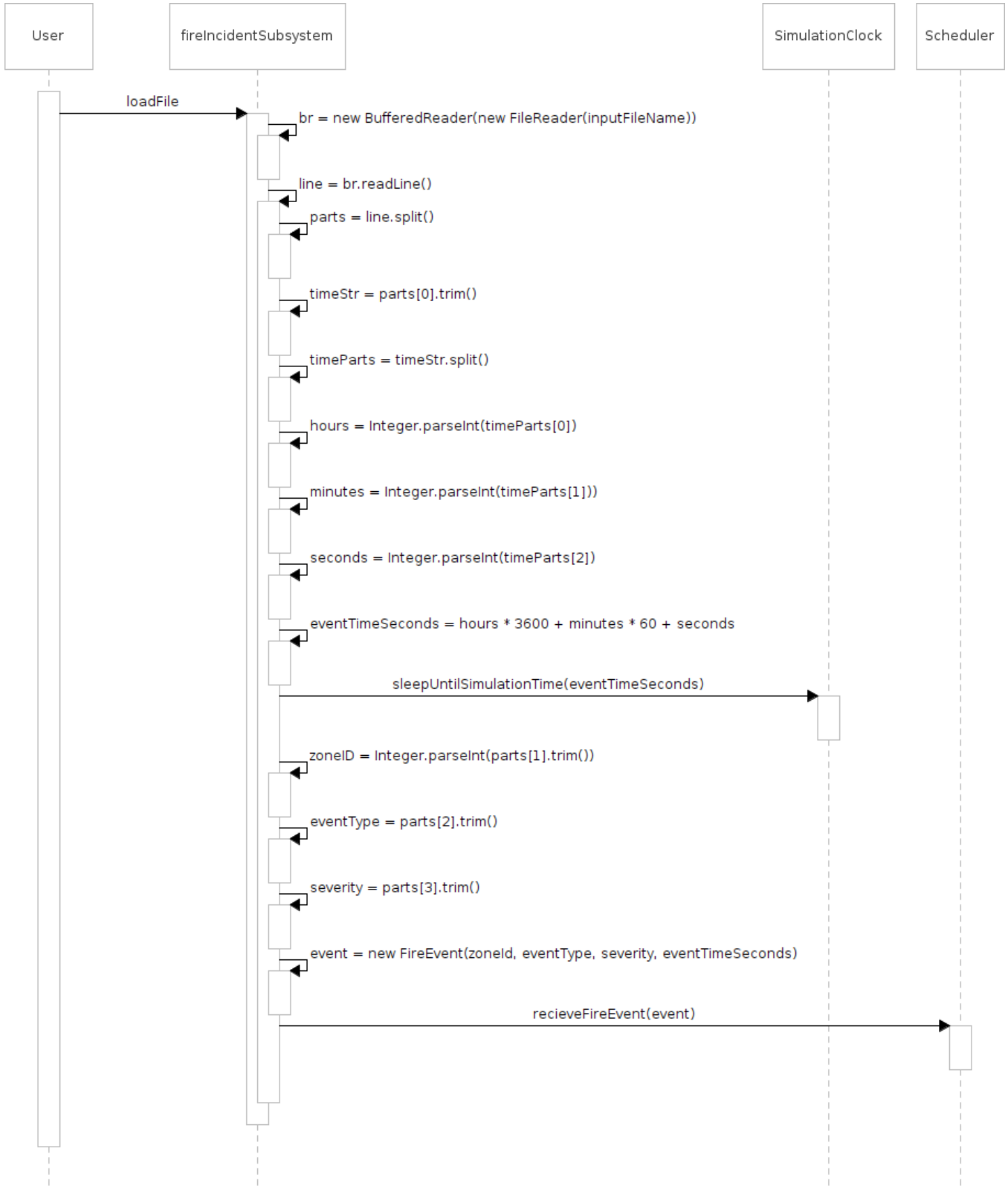

FAULT_HANDLING
entry / log("Enter FAULT_HANDLING")
exit / log("Exit FAULT_HANDLING")
fault resolved
[no queues, no active missions]
IDLE
entry / log("Enter IDLE")
exit / log("Exit IDLE")
fault resolved
[queues not empty]
fault resolved
[active missions exist]
receiveFireEvent()
(first fire arrives)
DISPATCHING
entry / log("Enter DISPATCHING")
exit / log("Exit DISPATCHING")
fault resolved
[refilling drones exist]
missionCompleted()
[last drone, queues empty]
requestMission()
[queues not empty]
or receiveFireEvent()
receiveFireEvent()
(new fire while drones active)
requestMission()
[queues empty and activeMissionCount>0]
droneRefilling()
droneRefillComplete()
[last refill, no active missions, queues empty]
MONITORING
entry / log("Enter MONITORING")
exit / log("Exit MONITORING")
missionCompleted()
[still active drones, queues empty]
droneRefillComplete()
[last refill, queues not empty]
droneRefilling()
droneRefillComplete()
[last refill, active missions exist, queues empty]
droneRefilling()
REFILLING
entry / log("Enter REFILLING")
exit / log("Exit REFILLING")
droneRefillComplete()
[still refilling drones]

# Sequence Diagrams:

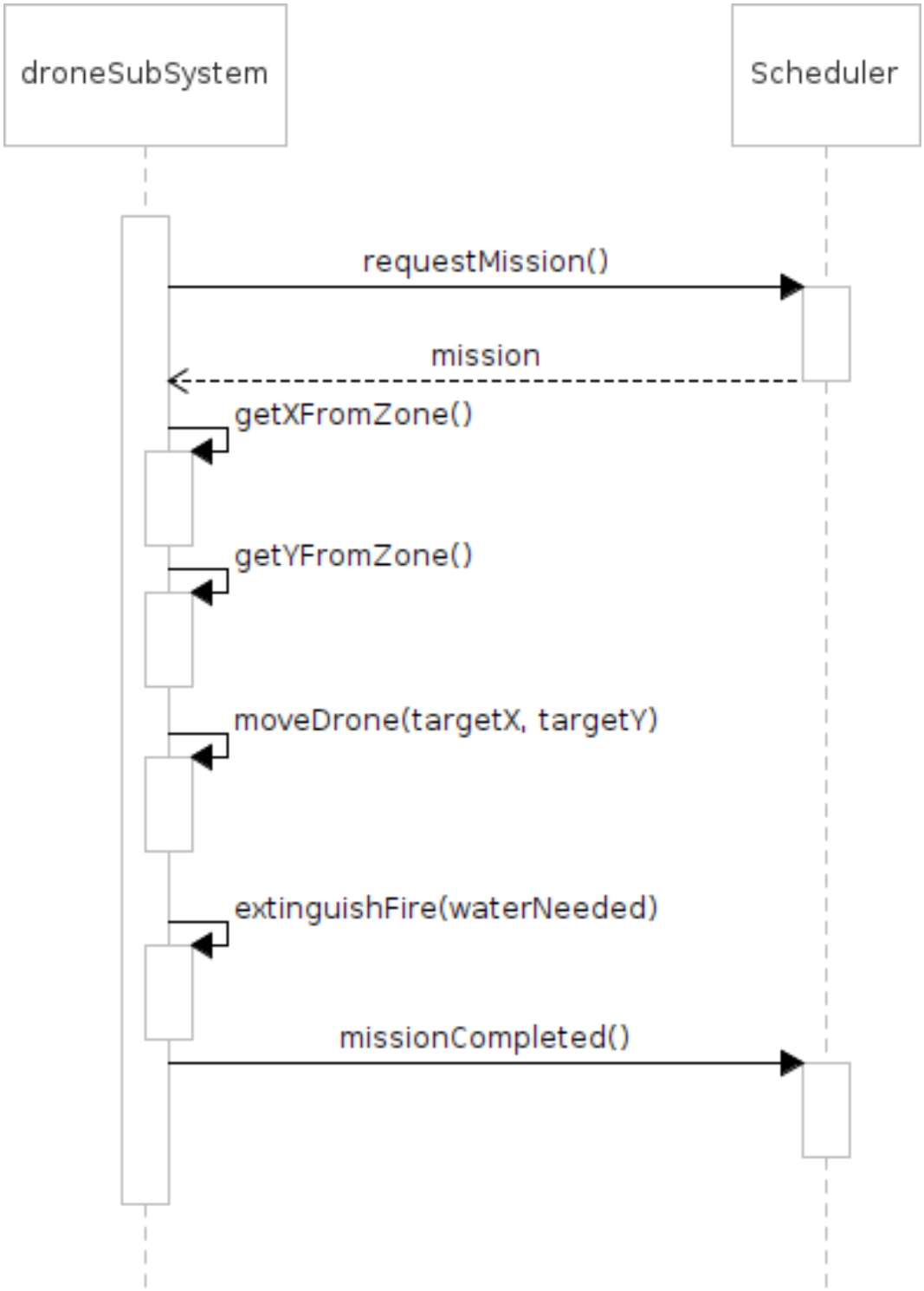
## Entire Program



Fire Subsystem:



Drone Subsystem



# Detailed Setup Instructions

1. Run the Scheduler main class, which will open the GUI window. You can load zone and fire event files from the GUI, or use the terminal flags below. java -cp out/production/Project SchedulerMain
2. Registers drones (e.g. four with IDs 1–4) with the Drone Subsystem. Add or remove IDs to change swarm size. java -cp out/production/Project DroneMain 1 2 3 4
3. Start the Fire Incident Subsystem and either wait for input via GUI, or pass files directly.
   java -cp out/production/Project FireIncidentMain
   java -cp out/production/Project FireIncidentMain --file src/fire_events.csv --zones src/zones.csv

**NOTE: Ensure CSV files are in the src folder (or provide an absolute path).**

## CSV File Formats

Fire events CSV (src/fire_events.csv):

```
Time, ZoneID, EventType, Severity
00:00:00, 1, FIRE_EVENT, HIGH
00:05:00, 2, FIRE_EVENT, MODERATE
00:25:00, 1, DRONE_STUCK, NONE
00:42:00, 2, NOZZLE_FAULT, NONE
```

- EventType: FIRE_EVENT (or FIRE) for fires; DRONE_STUCK or NOZZLE_FAULT for faults
- Severity: HIGH / MODERATE / LOW for fire events; NONE (or blank) for faults
- For fault rows the ID column is the drone ID, not a zone ID

Zone definitions CSV (src/zones.csv):

```
ZoneID, ZoneStart, ZoneEnd
1, (0, 0), (450, 450)
2, (450, 0), (900, 450)
3, (0, 450), (450, 900)
4, (450, 450), (900, 900)
```

Coordinates are in metres. Internally converted to grid cells at 30 m per cell (e.g. 450 m = 15 cells). Each zone must be a rectangle at least 90 m wide and 90 m tall (= 3 grid cells minimum per dimension).

## How To Run Tests

All unit tests relevant to iteration 5 are located in the test/Iteration 5/ directory. The tests verify the behavior of the Scheduler, DroneSubsystem, and FireEvent classes, including priority handling, partial mission assignment, and state transitions.

**Run all tests in the Iteration 5 package:**

1. In the Project tool window, navigate to test/Iteration 5.
2. Right-click on the package (or on any test class) and select Run 'Tests in 'Iteration 2'' (or Run All Tests).

**Run a single test class:**

- Open the desired test class (e.g., BatteryTest.java).
- Click the green triangle in the gutter next to the class declaration or any individual test method, then select Run 'SchedulerTest'.

# Results From Measurements

Here is the zone by zone metrics:

Total fires: 41
Avg Response Time: 20m 06s
Avg Extinguish Time:28m 29s

| Zone | Response Time | Extinguish Duration | Zone | Response Time | Extinguish Duration |
|---|---|---|---|---|---|
| Zone 1 | 6m 40s | 13m 30s | Zone 1 | 6m 40s | 11m 20s |
| Zone 2 | 26m 40s | 31m 20s | Zone 3 | 21m 40s | 28m 30s |
| Zone 3 | 21m 40s | 28m 30s | Zone 4 | 0s | 36m 50s |
| Zone 1 | 0s | 4m 40s | Zone 2 | 20m 00s | 26m 50s |
| Zone 4 | — | 30m 00s | Zone 5 | 35m 00s | 44m 10s |
| Zone 2 | 0s | 36m 50s | Zone 3 | 21m 40s | 28m 40s |
| Zone 5 | 51m 00s | 55m 40s | Zone 1 | 6m 40s | 13m 40s |
| Zone 4 | 0s | 26m 30s | Zone 4 | 30m 00s | 39m 10s |
| Zone 1 | 6m 30s | 11m 20s | Zone 2 | 26m 40s | 31m 20s |
| Zone 5 | 0s | 4m 40s | Zone 5 | 35m 00s | 44m 10s |
| Zone 3 | 21m 40s | 30m 50s | Zone 4 | 30m 00s | 39m 10s |
| Zone 1 | 6m 40s | 15m 40s | Zone 1 | 6m 30s | 15m 40s |
| Zone 4 | 29m 50s | 39m 00s | Zone 2 | 26m 40s | 31m 30s |
| Zone 3 | 21m 40s | 30m 40s | Zone 3 | 21m 30s | 30m 40s |
| Zone 2 | 26m 40s | 35m 40s | Zone 1 | 6m 40s | 11m 20s |
| Zone 5 | 35m 00s | 44m 00s | Zone 5 | 28m 10s | 39m 40s |
| Zone 3 | 21m 40s | 28m 30s | Zone 4 | 23m 10s | 28m 00s |
| Zone 4 | 30m 00s | 36m 50s | Zone 2 | 26m 30s | 35m 40s |
| Zone 2 | 18m 20s | 23m 00s | Zone 1 | 6m 40s | 13m 30s |
| Zone 5 | 35m 00s | 44m 00s | Zone 3 | 21m 30s | 30m 40s |
| Zone 1 | 6m 40s | 11m 20s | Zone 5 | 34m 50s | 41m 50s |

```
Drone Performance Metrics:
Drone       Missions     FlightTime   IdleTime     Recharges
Drone 1     8            3h 23m 10s   2h 40m 30s   6
Drone 2     8            2h 59m 40s   3h 07m 10s   7
Drone 3     8            3h 26m 00s   2h 47m 30s   4
Drone 4     7            3h 28m 10s   2h 37m 10s   5
Drone 5     7            3h 10m 20s   3h 01m 40s   5
Drone 6     1            2m 00s       56m 00s      0
Drone 7     4            2h 23m 10s   4h 25m 20s   4
Drone 8     3            1h 10m 00s   7h 00m 10s   2
Drone 9     1            15m 40s      8h 49m 30s   1
Drone 10    0            0s           9h 21m 50s   0
Drone 11    0            0s           9h 21m 50s   0
Drone 12    0            0s           9h 21m 50s   0
Drone 13    0            0s           9h 21m 50s   0
Drone 14    0            0s           9h 21m 50s   0
Drone 15    0            0s           9h 21m 50s   0
Drone 16    0            0s           9h 21m 50s   0
Drone 17    0            0s           9h 21m 50s   0
Drone 18    0            0s           9h 21m 50s   0
Drone 19    0            0s           9h 21m 50s   0
Drone 20    0            0s           9h 21m 50s   0

Avg drone idle time: 6h 55m 15s

Overall Simulation Metrics:
Total simulation time:  9h 21m 50s
First fire → last out:  8h 33m 10s
Total fires detected:   41
Total fires extinguished: 41
```

# Reflection

The simulation demonstrates that the system is highly effective in achieving its primary objective, as all 41 fires were successfully extinguished. However, this success comes with notable inefficiencies. Response times vary significantly across zones, with some areas receiving rapid attention while others experience considerable delays. This suggests uneven resource distribution and highlights gaps in coverage.

Drone performance further reinforces this imbalance. A small number of drones handled the majority of missions, while many remained inactive for nearly the entire simulation. The high average idle time indicates that available resources were not utilized to their full potential. As a result, some drones were likely overburdened while others contributed little to overall performance.

Overall, while the system is reliable, it is not optimized. Improving drone allocation and dispatch strategies could reduce response times, balance workload across the fleet, and make better use of available resources.